

彩讯科技-前端面经

彩讯前端一面

1. 对React18有了解吗？有什么新特性？，既然你提到了并发模式，那原理是什么？如何开启并发模式？
2. React批处理机制？
3. 平时对组件和代码是怎么样优化的？
4. 代码优化和组件优化有什么原则吗？
5. 什么情况下需要封装组件？
6. 你们每个函数代码行数有要求吗？按照什么要求来？
7. 项目中做了什么规范限制？ `Eslint` 是怎么配置的？
8. `useCallback` 和 `useMemo` 有什么区别？什么情况下需要用到这两个？
9. `useEffect` 使用的限制条件是什么？
10. 你选择TS的原因是什么？有什么优势？
11. TS的函数重载是什么？TS的可选链是什么？
12. 有没有用过 `Ant design` 的换肤？原理是什么？
13. CSS是异步加载的还是同步加载的？
14. 很多if嵌套你应该如何优化？
15. `redux-toolkit` 有什么看法？
16. `redux-thunk` 和 `redux-saga` 的区别？
17. `styled-components` 有用过吗？
18. 有没有用过富文本编辑器？
19. 现在要你作为项目负责人，有个新项目你如何处理？
20. 输入URL会发生什么？
21. 浏览器的渲染原理是什么？
22. DNS域名解析？
23. 三次握手，四次挥手是什么？为什么需要三次握手，四次挥手？SYN，ACK，FIN这些具体指的是什么东西（我回答二进制）？
24. 有个旧项目，后端重构了，导致接口地址和传参都变了，前端不想重构，如何处理？

参考答案：

1. 对React 18有了解吗？有什么新特性？既然你提到了并发模式，那原理是什么？如何开启并发模式？

React 18的新特性：

React 18 引入了一些重要的新特性：

- **并发模式（Concurrent Mode）**：这是React 18的核心升级。并发模式允许React在处理繁重的更新时，不会阻塞浏览器的主线程。它可以在用户交互和渲染之间进行更细粒度的调度，改善用户体验。
- **自动批处理（Automatic Batching）**：在React 18之前，React只会在合成事件处理程序中自动批处理更新。但在React 18中，任何地方的状态更新都会自动批处理。
- **useTransition 和 useDeferredValue**：这两个新的钩子（Hooks）帮助开发者管理UI中可中断的更新，比如用户输入时延迟渲染较重的组件。
- **新的 Suspense 功能**：React 18扩展了 `Suspense` 的功能，使其不仅能用于代码拆分（code-splitting），还可以用于数据获取。

并发模式的原理：

并发模式的核心是React的“时间切片”（Time Slicing）机制，这允许React在不阻塞主线程的情况下优先处理高优先级的任务。React通过调度器（Scheduler）将任务分成多个小的时间片，并根据任务的优先级来决定哪一部分任务需要先处理。这样可以在用户滚动、点击等高优先级事件发生时暂停低优先级的渲染任务，提升响应速度。

如何开启并发模式：

并发模式的启用方式较为简单，通过将React 18中提供的 `createRoot` API替换掉旧的 `ReactDOM.render` 来开启：

```
1 javascript
2 复制代码
3  import React from 'react';
4  import ReactDOM from 'react-dom/client';
5  import App from './App';
6
7  const root = ReactDOM.createRoot(document.getElementById('root'));
8  root.render(<App />);
```

这样React 18会自动启用并发模式，优化UI渲染和交互。

2. React批处理机制？

React 18中的批处理机制是自动批处理的扩展，它可以减少多次状态更新导致的重新渲染次数，从而提高性能。

旧机制：在React 18之前，批处理只在React合成事件（如点击、输入等）和生命周期方法内生效。如果在一个异步回调函数中多次更新状态，React不会批量处理这些更新。

新机制：在React 18中，所有的状态更新都自动批处理，包括异步回调函数。这意味着在一次事件循环中多次调用 `setState` 只会触发一次重新渲染，而不是多次。

3. 平时对组件和代码是怎么样优化的？

组件和代码优化的常见策略包括：

- **组件分割：**将复杂的大型组件拆分为多个小型、职责单一的组件，有助于复用代码，并提升渲染性能。
- **React.memo：**对于纯展示组件，可以使用 `React.memo` 来避免不必要的重新渲染。它只在组件的props变化时才重新渲染。
- **懒加载（Lazy Loading）：**对于不立即需要渲染的组件，通过 `React.lazy` 和 `Suspense` 来实现按需加载，从而减少初始加载时间。
- **减少不必要的依赖：**在 `useEffect`、`useCallback`、`useMemo` 等Hook的依赖数组中，仅加入必要的依赖，避免每次渲染都重新执行这些函数。
- **使用 `useCallback` 和 `useMemo`：**对于需要传递给子组件的回调函数和计算量较大的值，可以使用 `useCallback` 和 `useMemo` 进行缓存，避免子组件的无谓重渲染。

4. 代码优化和组件优化有什么原则吗？

代码优化原则：

- **可读性：**优化代码时，不仅要考虑性能，还要保持代码的可读性，避免过度优化导致代码难以维护。
- **早期返回（Early Return）：**减少嵌套，提升代码的简洁性和可读性。
- **单一职责原则（Single Responsibility Principle）：**确保每个函数、组件只做一件事，从而更易于测试和维护。
- **避免过早优化：**应在性能确实成为问题时才进行优化，过早优化可能会导致代码复杂性增加且收益较低。

组件优化原则：

- **拆分组件**：将复杂组件拆分为多个独立的小组件，既可以提高复用性，又可以提升渲染性能。
- **避免不必要的状态**：尽量减少组件中的状态，将状态尽可能提升至其共同父组件（状态提升）。
- **使用无状态组件（Stateless Components）**：对于仅依赖props的组件，使用无状态组件，减少不必要的状态管理。
- **使用React.memo**：对纯展示组件使用 `React.memo` 包裹，避免因父组件的重新渲染而导致子组件的不必要重新渲染。

5. 什么情况下需要封装组件？

封装组件的情况包括：

- **重复使用**：当多个地方需要相同的UI或者逻辑时，封装组件可以提高代码的复用性，减少重复代码。
- **提高可维护性**：封装组件使得代码更加模块化，容易测试和维护。
- **复杂逻辑抽象**：当某个UI包含复杂的交互逻辑时，通过封装将其独立出来，可以让父组件更加简洁。
- **跨项目复用**：在多个项目中需要复用时，将组件封装成独立的库或包。

6. 你们每个函数代码行数有要求吗？按照什么要求来？

一般来说，函数的代码行数应该保持简短，通常不超过50行，这是为了遵循单一职责原则（Single Responsibility Principle）。具体要求可能因团队的代码规范而异：

- **单一职责**：每个函数应该只做一件事。如果一个函数过长，可能意味着它在做多件事情，可以将其拆分为多个函数。
- **可读性**：短小的函数更易于理解和测试，也更容易复用。
- **避免过度拆分**：虽然短函数是好的实践，但过度拆分也可能导致代码过于分散，影响可读性。因此，要在简洁和功能完备之间取得平衡。

7. 项目中做了什么规范限制？Eslint是怎么配置的？

项目规范限制通常涵盖以下几个方面：

- **代码风格**：使用Eslint配置代码风格规则，比如缩进、引号类型、分号使用等，以保持代码的一致性。
- **提交信息规范**：使用 `commitlint` 等工具来规范Git提交信息，确保提交记录的可读性和可追溯性。
- **组件命名和目录结构**：对组件的命名和目录结构做出规范，确保项目的组织结构清晰。
- **代码审查**：通过代码审查流程来保证代码质量，确保每次合并的代码都经过团队的认可。

Eslint配置：

Eslint的配置一般包括以下几个方面：

- **代码风格规则**：例如Airbnb风格、Prettier集成，统一缩进、行尾、引号、分号等。
- **React和JSX规则**：如防止React未定义、禁止使用无用的 `jsx-boolean-value` 等。
- **TypeScript规则**：如果项目使用TypeScript，还需要配置 `@typescript-eslint`，以确保TS的类型检查与代码风格检查相结合。

配置示例：

```
1 json
2 复制代码
3 {"extends": ["eslint:recommended", "plugin:react/recommended",
  "plugin:@typescript-eslint/recommended", "airbnb", "prettier"], "plugins":
  ["react", "@typescript-eslint"], "rules": {"no-unused-vars": "warn", "react/prop-
  types": "off", "@typescript-eslint/no-unused-vars": ["error"]}, "parserOptions":
  {"ecmaVersion": 2020, "sourceType": "module", "ecmaFeatures": {"jsx":
  true}}, "settings": {"react": {"version": "detect"}}}
```

8. useCallback和useMemo有什么区别？什么情况下需要用到这两个？

`useCallback` 和 `useMemo` 的区别：

- **useCallback**： `useCallback` 会返回一个记忆化的回调函数，只有当依赖项发生变化时才会重新生成这个函数。常用于将回调函数传递给子组件，防止子组件的无意义重新渲染。
- **useMemo**： `useMemo` 会返回一个记忆化的计算结果，

，只有当依赖项发生变化时才会重新计算该结果。常用于优化计算量较大的操作，防止在每次渲染时重复计算。

什么时候使用 `useCallback` 和 `useMemo`：

- **useCallback 的使用场景：**
 - 当你有一个回调函数（比如点击事件处理函数）需要传递给子组件时，如果这个回调函数没有使用 `useCallback`，每次父组件重新渲染时，都会生成一个新的函数引用，这可能导致子组件的无谓重新渲染。通过 `useCallback`，你可以确保回调函数的引用在依赖项未变化时保持不变，从而避免子组件的重新渲染。

```
1 javascript
2 复制代码
3 const handleClick = useCallback(() => {console.log('Button clicked');
```

```
4  }, []);
```

- **useMemo 的使用场景：**

- 当你有一个计算量较大的操作或复杂的数据处理逻辑，并且这个逻辑的结果依赖于一些特定的值时，可以使用 `useMemo` 来缓存这个结果，只有当依赖项发生变化时才重新计算，以提高性能。

```
1 javascript
2 复制代码
3  const sortedData = useMemo(() => {return data.sort((a, b) => a.value -
    b.value);
4  }, [data]);
```

- 总的来说，`useCallback` 用于缓存函数，`useMemo` 用于缓存计算结果，都是为了优化性能，减少不必要的渲染或计算。

9. useEffect使用的限制条件是什么？

`useEffect` 是React的一个副作用钩子，用来在函数组件中处理副作用（如数据获取、订阅、手动修改DOM等）。使用 `useEffect` 时需要注意以下几个限制条件：

- **依赖数组的正确使用：** 依赖数组决定了 `useEffect` 的执行时机。如果依赖数组中的变量发生变化，`useEffect` 就会重新执行。因此，依赖数组应准确包含所有在副作用中使用的外部变量，避免遗漏导致的潜在问题或不必要的重新执行。如果依赖数组为空，`useEffect` 只会在组件挂载和卸载时执行。
- **副作用的清理：** 如果 `useEffect` 返回一个函数，这个函数会在组件卸载时执行，用于清理副作用（例如清除定时器、取消订阅等）。确保在需要清理的副作用中返回清理函数，避免内存泄漏。

```
1 javascript
2 复制代码
3  useEffect(() => {const timer = setInterval(() => {console.log('Tick');
4    }, 1000);
5  return () => clearInterval(timer); // 清理定时器
6  }, []);
```

- **避免在渲染阶段引入副作用：** `useEffect` 在浏览器完成布局与绘制后才会执行，这保证了副作用不会阻塞页面渲染。因此，不要在渲染阶段（即组件的主函数体内）引入副作用，确保副作用只在 `useEffect` 中处理。

- **条件副作用：**如果某些副作用只在满足特定条件时执行，可以将这些条件作为依赖项放入依赖数组中，而不是在 `useEffect` 内部使用条件语句来包裹副作用逻辑。

10. 你选择TS的原因是什么？有什么优势？

选择TypeScript的原因：

TypeScript (TS) 是一种在JavaScript的基础上添加了静态类型检查的语言，越来越多的项目选择使用TS，因为它可以提高代码的质量、可维护性和开发效率。

TypeScript的优势：

- **类型安全：**TypeScript通过静态类型检查，能够在开发时捕获很多潜在的错误，减少了运行时错误的发生。类型系统使得代码的意图更加明确，易于理解和维护。
- **增强的开发工具支持：**TypeScript提供了更好的代码自动补全、类型推导和重构工具支持，这些特性提高了开发效率，特别是在大型代码库中。
- **代码可读性和文档化：**类型注解作为代码的一部分，天然地提升了代码的可读性，并且可以自动生成API文档。
- **社区和生态系统支持：**TypeScript已经成为前端开发的主流选择，拥有庞大的社区支持，几乎所有的JavaScript库都提供了TypeScript类型定义。
- **更好的可维护性：**对于大型项目或长期维护的项目，TypeScript的类型检查帮助开发团队更好地理解和维护代码，减少了因接口变化而引发的错误。

11. TS的函数重载是什么？TS的可选链是什么？

TypeScript的函数重载：

TypeScript支持函数重载，即可以为同一个函数定义多个不同的调用签名，根据传入的参数类型或数量来决定实际调用哪个签名。这在需要处理多种不同参数组合的函数中非常有用。

```
1 typescript
2 复制代码
3 function add(a: number, b: number): number;
4 function add(a: string, b: string): string;
5 function add(a: any, b: any): any {return a + b;
6 }
7
8 const result1 = add(1, 2); // 输出: 3const result2 = add("Hello, ", "world!");
   // 输出: Hello, world!
```

TypeScript的可选链 (Optional Chaining) :

可选链（`?.`）是TypeScript 3.7引入的语法特性，用来安全地访问嵌套的对象属性，避免 `null` 或 `undefined` 导致的错误。它允许在访问对象属性时，如果前面的属性为 `null` 或 `undefined`，则表达式返回 `undefined`，而不会抛出错误。

```
1 typescript
2 复制代码
3 const user = {name: "Alice",address: {city: "Wonderland"
4   }
5 };
6
7 const city = user?.address?.city; // 输出: "Wonderland"const street =
  user?.address?.street; // 输出: undefined 而非抛出错误
```

12. 有没有用过Ant Design的换肤？原理是什么？

Ant Design的换肤功能可以通过修改Less变量或动态注入CSS来实现。Ant Design默认使用Less预处理器编写样式，通过更改Less变量的值，可以动态调整主题的颜色、字体等样式。

原理：

1. **修改Less变量：**Ant Design将其所有的样式都基于Less变量定义。通过修改这些变量，你可以自定义主题颜色等。例如，可以通过覆盖 `@primary-color` 变量来改变全局的主题色。
2. **动态主题切换：**通过将Less变量编译成CSS，然后动态注入到页面中，可以实现应用内的换肤功能。例如，可以通过一个按钮来切换不同的主题。
3. 这个过程通常需要配合 `less-loader` 和 `antd-theme-webpack-plugin` 等工具来进行动态样式编译和加载。

13. CSS是异步加载的还是同步加载的？

CSS的加载是同步的。当浏览器遇到 `<link>` 标签引入的外部CSS文件时，会停止HTML的解析，等待CSS文件加载和解析完成后再继续处理DOM的渲染。这是因为CSS样式会影响页面的布局和渲染，必须在渲染DOM之前应用样式。

- **CSS阻塞渲染：**浏览器在解析HTML时，如果遇到外部CSS文件，会阻塞渲染，直到CSS文件加载和解析完成。为了优化性能，可以考虑使用 `media` 属性，异步加载非关键的CSS。
- **关键渲染路径优化：**为了减少CSS加载对页面渲染的阻塞，可以将关键CSS内联，或者使用 `preload` 来提前加载关键CSS文件。

14. 很多if嵌套你应该如何优化？

多层嵌套的 `if` 语句会导致代码的可读性和维护性下降，可以通过以下方式进行优化：

- **早期返回 (Early Return)**：将不符合条件的情况尽早返回，减少嵌套层级。例如，将负面条件判断放在前面，可以减少主逻辑的嵌套。

```
1 javascript
2 复制代码
3  if (!user) {return 'User not found';
4  }
5
6  if (!user.isActive) {return 'User is not active';
7  }
8
9  // 主逻辑代码
```

- **使用逻辑运算符**：对于简单的条件判断，可以使用逻辑运算符代替 `if` 语句，例如 `&&` 和 `||`。

```
1 javascript
2 复制代码
3  const isAdmin = user && user.role === 'admin';
```

- **将条件逻辑提取到函数**：如果条件判断较复杂，可以将判断逻辑提取到一个单独的函数或使用策略模式，提升主逻辑的可读性。

```
1 javascript
2 复制代码
3  function isUserEligible(user) {return user && user.isActive && user.role ===
   'admin';
4  }
5
6  if (isUserEligible(user)) {// 主逻辑代码
7  }
```

- ****使用 `switch` 语**

使用 `switch` 语句：对于某些需要多重分支处理的情况，`switch` 语句可能比多个嵌套的 `if` 更清晰。

```
1 javascript
2 复制代码
3  switch (user.role) {case 'admin':// admin逻辑break;case 'editor':// editor逻辑
   break;default:// 默认逻辑
```

15. redux-toolkit有什么看法？

Redux Toolkit是Redux官方推荐的使用方式，旨在简化Redux的使用，减少样板代码，并提供一系列工具来提升开发体验。

优点：

- **简化配置：** Redux Toolkit 提供的 `configureStore` 可以轻松地创建Redux store，自动集成了中间件和开发者工具扩展（DevTools）。
- **减少样板代码：** 通过 `createSlice`，Redux Toolkit将action creators和reducers结合在一起，减少了传统Redux中的样板代码。
- **内置的 `immer` 库：** Redux Toolkit内置了 `immer`，使得编写不可变状态更新变得简单直观，无需手动处理深拷贝。
- **自动生成类型安全的action和reducers：** 对于TypeScript用户，Redux Toolkit能够自动生成类型安全的action creators和reducers，提升了开发效率。
- **推荐的最佳实践：** Redux Toolkit集成了社区公认的最佳实践，减少了由于错误配置或不熟悉Redux而导致的问题。

缺点：

- **学习曲线：** 虽然Redux Toolkit简化了Redux的使用，但对于初学者来说，仍然需要一些时间来理解 `createSlice`、`createAsyncThunk` 等新概念。
- **灵活性限制：** 由于Redux Toolkit封装了很多细节，对于一些高级用例，开发者可能需要花更多时间来理解其底层实现。

总体而言，Redux Toolkit极大地提升了Redux的开发体验，是一个非常推荐的工具，尤其是对于新项目或需要维护大型应用的开发者。

16. redux-thunk和redux-saga的区别？

redux-thunk和**redux-saga**都是处理Redux异步逻辑的中间件，但它们的实现方式和使用场景有所不同。

redux-thunk：

- **实现简单：** `redux-thunk` 允许你在action creator中返回一个函数，而不是action对象。这个函数可以接收 `dispatch` 和 `getState` 作为参数，从而可以在函数内部处理异步逻辑，并在异步操作完成后再派发action。
- **适合简单的异步逻辑：** `redux-thunk` 非常适合处理简单的异步请求，如基本的 `GET`、`POST` 操作。

- **直接操作 `dispatch`**：`redux-thunk` 使你可以直接在异步操作中调用 `dispatch`，这是它的核心特点。

```
1 javascript
2 复制代码
3 const fetchUser = (userId) => (dispatch) => {dispatch({ type:
  'FETCH_USER_REQUEST' });fetch(`/api/users/${userId}`)
4   .then((response) => response.json())
5   .then((data) => dispatch({ type: 'FETCH_USER_SUCCESS', payload: data }))
6   .catch((error) => dispatch({ type: 'FETCH_USER_FAILURE', error }));
7 };
```

redux-saga:

- **功能强大**：`redux-saga` 基于ES6的生成器函数（generators），通过 `take`、`call`、`put`、`select` 等effect创建器来描述异步逻辑，提供了更强大的控制流，特别是对于复杂的异步操作，如顺序执行、并发控制、取消任务等。
- **适合复杂的异步逻辑**：`redux-saga` 非常适合处理复杂的异步流程和副作用管理，如并发请求、延迟任务、条件控制流、重试机制等。
- **独立于React组件**：`redux-saga` 的副作用逻辑完全独立于React组件，因此更容易测试和维护。

```
1 javascript
2 复制代码
3 import { call, put, takeEvery } from 'redux-saga/effects';
4
5 function* fetchUser(action) {try {const user = yield call(Api.fetchUser,
  action.payload.userId);yield put({ type: 'FETCH_USER_SUCCESS', user: user });
6   } catch (e) {yield put({ type: 'FETCH_USER_FAILURE', message: e.message });
7   }
8 }
9
10 function* mySaga() {yield takeEvery('FETCH_USER_REQUEST', fetchUser);
11 }
```

选择依据:

- **redux-thunk**适合处理简单的异步逻辑，且较为容易上手。如果你的项目中异步操作较为简单，可以考虑使用 `redux-thunk`。
- **redux-saga**适合复杂的异步场景，尤其是需要精确控制副作用的执行流程时。`redux-saga` 的学习曲线较陡，但在处理复杂异步逻辑时更为强大和灵活。

17. styled-components有用过吗？

styled-components是一个流行的CSS-in-JS库，它允许你在JavaScript中直接编写CSS，并将样式与组件紧密耦合。使用 `styled-components` 可以创建封装良好的UI组件，样式和逻辑完全在一个文件中。

主要特性：

- **标签模板字面量：** `styled-components` 使用ES6的标签模板字面量语法，使得编写样式直观简洁。

```
1 javascript
2 复制代码
3 import styled from 'styled-components';
4
5 const Button = styled.button`
6   background: ${props => props.primary ? 'blue' : 'white'};
7   color: ${props => props.primary ? 'white' : 'black'};
8   padding: 10px;
9   border: none;
10  border-radius: 5px;
11 `;
```

- **动态样式：**支持根据props动态设置样式，提升了组件的灵活性。
- **自动生成唯一类名：** `styled-components` 会为每个组件生成一个唯一的类名，避免样式冲突。
- **CSS嵌套：**支持Sass风格的CSS嵌套语法，使样式层次更加清晰。
- **主题支持：** `styled-components` 内置了主题支持，可以通过 `ThemeProvider` 为应用提供一致的主题样式。

使用感受：

`styled-components` 将样式与组件结合，减少了全局CSS的复杂性，特别是在大型应用中，能够更好地管理和维护样式。对于需要动态样式或者根据主题切换的应用，`styled-components` 非常适合。

18. 有没有用过富文本编辑器？

富文本编辑器允许用户在进行复杂的文本编辑和格式化操作，如加粗、斜体、插入链接、图片等。常见的富文本编辑器包括 `Draft.js`、`Quill.js`、`TinyMCE`、`CKEditor` 等。

使用经验：

- **Draft.js：**由Facebook开发，是React生态中的流行富文本编辑器。`Draft.js` 提供了高度可定制的API，可以实现自定义的编辑行为和内容块。它较为灵活，但需要较多的配置和定制工作。

- **Quill.js**: 一个轻量级且开箱即用的富文本编辑器。`Quill.js` 的API简洁直观, 非常适合快速集成富文本编辑功能, 且支持丰富的扩展和模块。
- **TinyMCE/CKEditor**: 这两个是历史悠久的富文本编辑器, 功能非常强大, 支持复杂的文本编辑功能, 适合对文本编辑要求非常高的项目。

选型依据:

- 如果你需要一个轻量级且快速集成的解决方案, `Quill.js` 是不错的选择。
- 如果你的项目对文本编辑有高度定制化的需求, `Draft.js` 提供了极大的灵活性。
- 如果你需要一个功能齐全、易于配置的编辑器, `TinyMCE` 或 `CKEditor` 是成熟的选择。

19. 现在要你作为项目负责人, 有个新项目你如何处理?

作为项目负责人, 处理新项目时的步骤如下:

1. 需求分析:

- 与客户或产品经理深入讨论, 明确项目的需求和目标。
- 确定项目的范围、优先级和预期交付成果。

2. 技术选型:

- 根据项目需求选择合适的技术栈。考虑因素包括项目的规模、团队的技术能力、性能要求、未来的可扩展性等。

3. 项目计划:

- 制定详细的项目计划, 包括任务分解、时间表、里程碑和资源分配。
- 使用项目管理工具 (如JIRA、Trello) 来跟踪任务和进度。

4. 团队组建与分工:

- 根据项目的需求, 组建开发团队, 并明确每个成员的角色和职责。
- 确保团队内沟通顺畅, 定期组织会议以确保项目进展。

5. 开发流程与规范:

- 确立代码规范、版本控制策略、CI/CD流程、代码审查

9. 输入URL会发生什么?

当在浏览器中输入一个URL后, 会发生一系列的过程来加载和显示该URL对应的网页。具体过程如下:

1. **URL解析**: 浏览器首先会解析输入的URL。URL通常由协议 (如HTTP、HTTPS)、域名 (或IP地址)、端口号 (如果未指定, 默认为协议的默认端口)、路径 (指定服务器上的资源位置)、查询参数和片段标识符组成。浏览器会将这些部分分解并提取出来, 以便后续的操作。

2. DNS解析：如果输入的URL中包含了域名而非IP地址，浏览器会进行DNS解析，将域名解析成相应的IP地址。DNS解析通过向域名服务器发送查询请求，并接收服务器返回的IP地址来完成。一旦浏览器获取了目标服务器的IP地址，它就可以通过该地址与服务器建立连接。
3. 建立TCP连接：浏览器使用HTTP协议或HTTPS协议与服务器通信。如果是HTTP协议，浏览器会尝试与服务器的默认HTTP端口（通常是80）建立TCP连接；如果是HTTPS协议，浏览器会尝试与服务器的默认HTTPS端口（通常是443）建立加密的TLS连接。这个过程通常涉及“三次握手”，即浏览器向服务器发送一个连接请求，服务器确认请求并回复，最后浏览器再次确认服务器的回复。
4. 发送HTTP请求：一旦TCP连接建立完成，浏览器会向服务器发送HTTP请求。这个请求包含了之前解析得到的URL、请求方法（GET、POST等）、请求头部（包含浏览器和客户端的信息、所需的数据格式等）以及请求体（对于POST请求，通常包含用户提交的数据）。
5. 服务器处理请求并返回响应：服务器收到浏览器发送的请求后，会根据请求的内容进行相应的处理。这可能涉及到从服务器上获取请求的资源（如HTML文件、图片、视频等），执行数据库查询、处理用户提交的数据等操作。处理完成后，服务器会生成一个HTTP响应，包含了响应状态码（指示请求的成功或失败）、响应头部（包含服务器信息、内容类型、缓存控制等）以及响应体（所请求资源的实际内容）。
6. 接收并渲染响应：浏览器接收到服务器返回的HTTP响应后，会根据响应的内容进行相应的处理。如果响应的内容是HTML，浏览器会解析HTML并构建DOM树，然后根据CSS样式信息构建渲染树，最终将DOM树和渲染树结合起来，展示给用户。如果响应的内容是其他资源（如图片、视频、Java文件等），浏览器会根据其内容类型进行相应的处理，并将其展示在页面上或执行相应的操作。
7. 断开连接：一旦浏览器完成了对响应的处理，它会关闭与服务器的TCP连接。在HTTP/1.1中，连接通常会保持一段时间以便于后续的请求，这被称为“持久连接”。在HTTP/2及更新的版本中，多个请求可以通过同一个连接并行处理，以提高性能。

10. 浏览器的渲染原理是什么？

浏览器的渲染原理涉及一系列复杂的过程，主要包括以下几个阶段：

1. 构建DOM树：浏览器无法直接理解和使用HTML，所以需要将HTML转换为浏览器能够理解的结构——DOM树。DOM树是由DOM元素及属性节点组成的。
2. 样式计算：样式计算的目的是为了计算出DOM节点中每个元素的具体样式。这个阶段大体可分为三步来完成：
 - 解析CSS，转换样式表中的属性值，使其标准化。
 - 计算DOM节点的具体样式。
 - 浏览器也是无法直接理解纯文本的CSS样式，所以当渲染引擎接收到CSS文本时，会执行一个转换操作，将CSS文本转换为浏览器可以理解的结构——styleSheets。

3. 布局：布局的目的是要计算出DOM树中可见元素的几何位置，例如节点的宽高、相对包含块的位置。布局树的每个节点都记录了x、y坐标和边框尺寸。DOM树还含有很多不可见的元素，比如head标签，还有使用了display:none属性的元素。所以在显示之前，还要额外地构建一棵只包含可见元素的布局树。
4. 分层：为了更加方便地实现页面中的一些复杂效果，如3D变换、页面滚动等，渲染引擎还需要为特定的节点生成专用的图层，并生成一棵对应的图层树（LayerTree）。浏览器页面实际上被分成了很多图层，这些图层叠加后合成了最终的页面。
5. 绘制：主线程会为每一个层单独产生绘制指令集，用于描述这一层的内容如何画出来。绘制列表只是用来记录绘制顺序和绘制指令的列表。当图层的绘制列表准备好之后，主线程会把该绘制列表提交（commit）给合成线程。
6. 合成：合成线程将每个图层栅格化，然后只需要把可视区的内容组合成一帧，展示给用户即可。

11. DNS域名解析？

DNS（Domain Name System，域名系统）是互联网中的一种分布式数据库系统，它用于将域名（例如www.example.com）转换为对应的IP地址（例如192.0.2.1），以便于计算机网络设备进行通信和数据传输。以下是DNS域名解析的详细介绍：

1. 定义与作用：DNS是互联网基础设施的重要组成部分，其主要作用是提供域名与IP地址之间的映射服务，使用户能够通过便于记忆的域名访问到全球范围内的网络资源。
2. 使用的协议及端口号：DNS使用两种主要的协议，即UDP（User Datagram Protocol）和TCP（Transmission Control Protocol），同时它们使用不同的端口号。
 - UDP协议：大多数DNS查询和响应都使用UDP协议。UDP协议在传输数据时不保证数据包的顺序和可靠性，但它速度快，适合用于DNS查询这种轻量级的数据交换。端口号是53。
 - TCP协议：DNS也可以使用TCP协议，通常用于大数据量的传输或者一些特殊情况下，比如DNS响应超过512字节时（因为UDP有时会被截断）。端口号同样是53。
3. 域名体系结构：DNS服务协议采用类似目录树的层次结构记录域名与IP地址的映射对应关系，形成一个分布式的数据库系统。
4. 查询方式：
 - 递归查询：DNS服务器接收到客户机请求，必须使用一个准确的查询结果回复客户机。如果DNS服务器本地没有存储查询DNS信息，那么该服务器会询问其他服务器，并将返回的查询结果提交给客户机。
 - 迭代查询：当客户机发送查询请求时，DNS服务器并不直接回复查询结果，而是告诉客户机另一台DNS服务器地址，客户机再向这台DNS服务器提交请求，依次循环直到返回查询的结果为止。
5. 域名解析过程：假定某客户机想获知域名为xxx.xyk.com主机的IP地址，域名解析的过程如下：
 - 客户机向其本地域名服务器发出DNS请求报文。

- 本地域名服务器收到请求后，查询本地缓存，若没有该记录，则以DNS客户的身份向根域名服务器发出解析请求。
- 根域名服务器收到请求后，判断该域名属于哪个域，将对应的顶级域名服务器的IP地址返回给本地域名服务器。
- 本地域名服务器向顶级域名服务器发出解析请求报文。
- 顶级域名服务器收到请求后，判断该域名属于哪个授权域名，因此将对应的授权域名服务器的IP地址返回给本地域名服务器。
- 本地域名服务器向授权域名服务器发起解析请求报文。
- 授权域名服务器收到请求后，将查询结果返回给本地域名服务器。
- 本地域名服务器将查询结果保存到本地缓存，同时返回给客户机。

12. 三次握手，四次挥手是什么？为什么需要三次握手，四次挥手？SYN，ACK，FIN这些具体指的是什么东西（二进制表示）？

三次握手和四次挥手是TCP（Transmission Control Protocol，传输控制协议）中用于建立连接和断开连接的机制。

三次握手的过程如下：

1. 第一次握手：客户端发送一个带有SYN（synchronize，同步）标志位的数据包给服务端。在TCP头部，SYN标志位用二进制表示为1时，表示这是一个连接请求报文段。此时，客户端进入SYN_SEND状态，等待服务器确认。
2. 第二次握手：服务端收到SYN报文后，如果同意建立连接，会发送一个SYN+ACK（同步+确认）报文作为应答，同时进入SYN_RECV状态。在TCP头部，SYN和ACK标志位同时用二进制表示为1时，表示这是对连接请求的确认，并同时发送一个自己的连接请求。
3. 第三次握手：客户端收到服务端的SYN+ACK报文后，会发送一个ACK报文作为应答，此时客户端和服务端都进入ESTABLISHED（已建立连接）状态，完成三次握手，开始传输数据。在TCP头部，ACK标志位用二进制表示为1时，表示这是一个确认报文段。

为什么需要三次握手：

三次握手主要是为了确保连接的可靠性和准确性。通过三次握手，客户端和服务端能够确认双方的接收能力和发送能力是否正常，并同步连接双方的序列号和确认号，为后续的数据传输做准备。这可以防止已失效的请求报文段突然又传送到了服务端而造成连接的误判，以及防止客户端发送SYN后由于网络原因没有收到服务端的SYN+ACK回复而导致服务端出现垃圾连接的情况。

四次挥手的过程如下：

1. 第一次挥手：客户端发送一个FIN（finish，结束）报文段给服务端，用来关闭客户端到服务端的数据传送。在TCP头部，FIN标志位用二进制表示为1时，表示这是一个断开连接的请求。此时客户端进入FIN_WAIT_1状态。
2. 第二次挥手：服务端收到FIN报文后，会发送一个ACK报文作为应答，表示同意关闭连接。此时服务端进入CLOSE_WAIT状态，等待自己的数据发送完毕。
3. 第三次挥手：服务端发送一个FIN报文给客户端，表示自己的数据已经发送完毕，同意关闭连接。此时服务端进入LAST_ACK状态。
4. 第四次挥手：客户端收到服务端的FIN报文后，会发送一个ACK报文作为应答，表示已经收到服务端的关闭连接请求。此时客户端进入TIME_WAIT状态，等待一段时间后（通常为2MSL，MSL是最长报文段寿命），确保服务端收到自己的ACK报文后，才进入CLOSED状态，断开连接。服务端收到ACK报文后，也进入CLOSED状态。

为什么需要四次挥手：

TCP连接是全双工的，即数据可以在两个方向上同时传输。因此，当一方想要断开连接时，需要发送一个FIN报文来关闭自己的发送通道，并等待对方发送一个FIN报文来关闭对方的发送通道。这样，双方都能确保自己的数据已经发送完毕，并且对方也已经知道要断开连接。因此，需要四次挥手来确保双方都能正确地关闭连接。

13. 有个旧项目，后端重构了，导致接口地址和传参都变了，前端不想重构，如何处理？

对于旧项目后端重构导致接口地址和传参变化，而前端不想进行重构的情况，可以尝试以下几种方法：

1. 使用代理服务器：
 - 搭建一个代理服务器，将前端请求的旧接口地址转发到新的接口地址。
 - 在代理服务器上处理传参的转换，确保新接口能够正确接收和处理。
2. 修改前端配置文件：
 - 如果前端使用了配置文件来管理接口地址和传参，可以直接修改配置文件中的相关信息。
 - 但这种方法需要确保前端代码中没有硬编码的接口地址和传参，否则仍然需要进行部分重构。
3. 使用后端适配层：
 - 在后端重构后的系统中，添加一个适配层来接收旧接口的请求。
 - 适配层负责将旧接口的请求转换为新接口的请求，并处理传参的转换。
 - 这种方法可以保持前端代码不变，但需要后端进行一定的开发工作。
4. 自动化生成接口文件：

- 如果后端接口项目迁移或重构导致前端需要修改大量接口文件，可以考虑使用自动化工具生成前端引用的接口文件。
- 通过在后端接口项目中配置一个系统，自动化生成包含示例和参数的接口文件，前端可以直接引入使用。

5. 评估重构的必要性：

- 在决定不进行前端重构之前，需要仔细评估重构的必要性和成本。
- 如果前端代码与后端接口的交互方式已经过时或存在严重问题，那么进行重构可能是必要的。
- 反之，如果前端代码只是需要简单地更新接口地址和传参，那么可以考虑使用上述方法进行处理。

需要注意的是，无论选择哪种方法，都需要确保前端与后端之间的通信能够正确地进行，并且不会出现数据丢失或错误处理的情况。同时，也需要考虑到未来的维护和扩展性，确保所选方案能够适应未来可能的变化。